

Design and Analysis of Algorithms

Problem Set 2 — Complete Solutions

1 DFS and Topological Ordering

Problem 1: Counting Shortest Paths in a DAG

Statement. $G = (V, E)$ is a DAG on n vertices and m edges, every edge length is a positive real number, and there is a unique vertex s such that every vertex of G is reachable from s . Let $N(v)$ be the number of shortest paths from s to v . Design an $O(m + n)$ time algorithm computing $N(v)$ for all $v \in V$, assuming every $N(v)$ is polynomial in n .

Why the polynomial assumption matters. We work in the standard word-RAM model, in which arithmetic (comparison, addition) on numbers occupying $O(\log n)$ bits costs $O(1)$ time. If some $N(v)$ were exponential in n , then $N(v)$ would need $\Omega(n)$ bits to be written down, so a single addition $N(v) \leftarrow N(v) + N(u)$ would cost $\Omega(n)$ time, and merely printing the answers would cost $\Omega(n^2)$ time in the worst case. No $O(m + n)$ bound is then possible. The hypothesis $N(v) = \text{poly}(n)$ guarantees that each count fits in $O(\log n)$ bits, so every arithmetic operation performed below is genuinely $O(1)$ and the claimed running time is honest.

Algorithm. Since G is a DAG, a topological order allows us to compute both the distance $D(v) = \delta(s, v)$ and the count $N(v)$ in a single forward sweep. Note that, because every vertex is reachable from s , the vertex s is the first vertex of *every* topological order of G .

Algorithm 1: CountShortestPaths(G, s)

Input: DAG $G = (V, E)$ with positive edge lengths w , source s reaching all of V .
Output: $D(v) = \delta(s, v)$ and $N(v)$ = number of shortest s - v paths, for all v .

```
1 Compute a topological order  $v_1, v_2, \dots, v_n$  of  $G$  by DFS (reverse order of finish times);  
   //  $O(m + n)$   
2  $D(s) \leftarrow 0, N(s) \leftarrow 1$ ;  
3 foreach  $v \neq s$  do  $D(v) \leftarrow \infty, N(v) \leftarrow 0$ ;  
4 foreach  $u = v_1, \dots, v_n$  in topological order do  
5   foreach  $\text{edge } (u, v) \in E$  do  
6     if  $D(u) + w(u, v) < D(v)$  then  
7        $D(v) \leftarrow D(u) + w(u, v)$ ;  
8        $N(v) \leftarrow N(u)$ ;  
9     else if  $D(u) + w(u, v) = D(v)$  then  
10       $N(v) \leftarrow N(v) + N(u)$ ;  
11 return  $D, N$ ;
```

Complexity. Topological sorting costs $O(m + n)$. The double loop examines each edge exactly once and performs $O(1)$ arithmetic per edge (by the assumption above). Total: $O(m + n)$.

Theorem 1.1. *At termination, $D(v) = \delta(s, v)$ and $N(v)$ equals the number of distinct shortest s - v paths, for every $v \in V$.*

Proof. We use strong induction on the position of a vertex in the topological order $v_1, \dots, v_n$, where $v_1 = s$.

Base case. For $v_1 = s$ we have $D(s) = 0 = \delta(s, s)$, and the only shortest path from s to s is the empty path, so $N(s) = 1$ is correct. Moreover s has no incoming edges (any in-edge would create a vertex preceding s in a topological order, contradicting that s is first), so these values are never modified.

Inductive step. Let $k \geq 2$ and assume the claim holds for $v_1, \dots, v_{k-1}$, in the following precise form: when the outer loop reaches v_j ($j < k$), the values $D(v_j), N(v_j)$ are already final and correct. Every edge $(u, v_k) \in E$ has $u = v_j$ with $j < k$, because the order is topological. Hence all in-edges of v_k are relaxed before the outer loop reaches v_k , and at the moment edge (v_j, v_k) is relaxed the tail value $D(v_j) = \delta(s, v_j)$ is correct and final.

Because all edge lengths are positive, every prefix of a shortest path is itself a shortest path (optimal substructure), so

$$\delta(s, v_k) = \min_{(u, v_k) \in E} (\delta(s, u) + w(u, v_k)),$$

and after all in-edges of v_k have been scanned the variable $D(v_k)$ holds exactly this minimum, i.e. $D(v_k) = \delta(s, v_k)$.

For the counts, let $M = \{(u, v_k) \in E : \delta(s, u) + w(u, v_k) = \delta(s, v_k)\}$ be the set of *minimizing* in-edges. Every shortest s - v_k path consists of a shortest s - u path followed by an edge $(u, v_k) \in M$, and conversely every such concatenation is a shortest s - v_k path. Distinct choices of the last edge or of the prefix give distinct paths, so

$$N(v_k) = \sum_{(u, v_k) \in M} N(u).$$

The algorithm computes exactly this sum: the “<” branch discards the contributions accumulated for a strictly larger tentative distance and restarts the sum, while the “=” branch adds the contribution of a further minimizing edge; contributions of non-minimizing edges are never added, since such an edge always yields a value strictly larger than the running minimum at the time it is scanned or strictly larger than the final value. Hence $N(v_k)$ is correct, completing the induction. $\square$

Problem 2: Realizability of Edge Classifications

Let $G = (V, E)$ be a directed graph without parallel edges and let $(u, v) \in E$. For each edge type we determine the exact condition on G under which *some* DFS forest classifies (u, v) as that type. Throughout we use the freedom of DFS to choose the roots of the forest and the order in which each adjacency list is scanned.

(a) Tree edge — always possible; no condition on G .

Claim 1.2. *For every edge $(u, v) \in E$ there is a DFS forest of G in which (u, v) is a tree edge.*

Proof. Start the DFS forest at u and scan v first in u ’s adjacency list. Nothing has been visited before u , so v is white when edge (u, v) is examined; DFS therefore recurses into v along this edge, which makes (u, v) a tree edge by definition. $\square$

(b) Forward edge — possible iff G has a $u \rightarrow v$ path of length ≥ 2 .

Claim 1.3. *There is a DFS forest of G in which (u, v) is a forward edge if and only if G contains a directed path from u to v consisting of at least two edges.*

Proof. ($\Leftarrow$) Let $u = x_0, x_1, \dots, x_k = v$ with $k \geq 2$ be such a path. Start the DFS at u and scan x_1 before v in u 's adjacency list. At the moment x_1 is discovered, the vertices $x_2, \dots, x_k$ are still white, so by the white-path theorem $v = x_k$ becomes a proper descendant of x_1 , hence a proper descendant of u , and the tree edge entering v is not (u, v) (it lies on the branch through $x_1 \neq v$). When u 's adjacency list is scanned further and (u, v) is examined, v is already discovered and is a proper descendant of u ; therefore (u, v) is classified as a forward edge.

($\Rightarrow$) If (u, v) is a forward edge of some DFS forest, then v is a proper descendant of u , so the tree path $u \rightarrow x_1 \rightarrow \dots \rightarrow x_k = v$ ($k \geq 1$) is a directed path of G from u to v . If $k = 1$ then $x_1 = v$, i.e. v is a child of u ; as G has no parallel edges, the tree edge into v would be the edge (u, v) itself, contradicting that (u, v) is classified as forward. Hence $k \geq 2$. $\square$

(c) Back edge — possible iff G has a $v \rightarrow u$ path.

Claim 1.4. *There is a DFS forest of G in which (u, v) is a back edge if and only if G contains a directed path from v to u (equivalently, (u, v) lies on a directed cycle).*

Proof. ($\Leftarrow$) Let $v = y_0, y_1, \dots, y_\ell = u$ be a path from v to u . Start the DFS at v and scan adjacency lists so that this path is followed immediately; since all its vertices are white at time $d[v]$, they become a chain of tree edges and u becomes a descendant of v , with v still gray when u is discovered. When (u, v) is examined, v is gray and is an ancestor of u , so (u, v) is a back edge.

($\Rightarrow$) If (u, v) is a back edge then v is an ancestor of u , and the corresponding chain of tree edges is a directed path from v to u in G . $\square$

(d) Cross edge — possible iff G has *no* $v \rightarrow u$ path.

Claim 1.5. *There is a DFS forest of G in which (u, v) is a cross edge if and only if there is no directed path from v to u in G .*

Proof. ($\Leftarrow$) Suppose v cannot reach u . Start the DFS at v . When the call $\text{DFS}(v)$ returns, exactly the vertices reachable from v have been visited; u is not among them, so u is still white and v is black with $f[v] < d[u]$. Restart the DFS at u . When (u, v) is examined, v is black; v is not a descendant of u (it finished before u was discovered) and not an ancestor of u (it was never gray while u was being explored). Hence (u, v) is a cross edge.

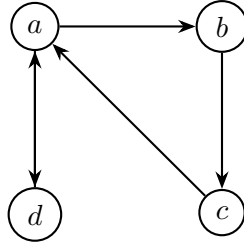
($\Rightarrow$) Suppose (u, v) is a cross edge, so v is neither an ancestor nor a descendant of u . By the parenthesis theorem their intervals $[d[\cdot], f[\cdot]]$ are disjoint, and a cross edge always points to an already finished vertex, so $f[v] < d[u]$. If v could reach u , then at time $d[v]$ there would be a white path from v to u , so u would be a descendant of v and in particular $d[u] < f[v]$, a contradiction. Hence v cannot reach u . $\square$

Problem 3: True/False on DFS and Strong Connectivity

(a) FALSE.

Claim 1.6. *If u and v are strongly connected, it need not be the case that one of them is an ancestor of the other in every DFS tree.*

Counterexample. Let $V = \{a, b, c, d\}$ with $E = \{a \rightarrow b, b \rightarrow c, c \rightarrow a, a \rightarrow d, d \rightarrow a\}$.



All four vertices are mutually reachable through a (for instance $b \rightarrow c \rightarrow a \rightarrow d$ and $d \rightarrow a \rightarrow b$), so V is a single strongly connected component; in particular b and d are strongly connected.

Run DFS from a with adjacency order $a : [b, d]$. We obtain $d[a] = 1$; tree edge $a \rightarrow b$, $d[b] = 2$; tree edge $b \rightarrow c$, $d[c] = 3$; edge $c \rightarrow a$ is a back edge; $f[c] = 4$, $f[b] = 5$; tree edge $a \rightarrow d$, $d[d] = 6$; edge $d \rightarrow a$ is a back edge; $f[d] = 7$, $f[a] = 8$.

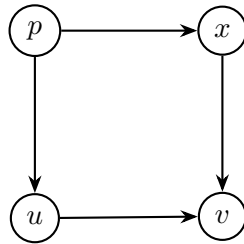
The intervals of b and d are $[2, 5]$ and $[6, 7]$: disjoint. Hence neither b nor d is an ancestor of the other, although they are strongly connected. $\square$

Remark 1.7. The correct related statement is weaker: the vertex of a strongly connected component that is discovered first is an ancestor of all other vertices of that component (here a is an ancestor of b, c, d). It does not extend to arbitrary pairs inside the component.

(b) FALSE.

Claim 1.8. *In a DAG, the existence of a path from u to v does not guarantee that u is an ancestor of v in every DFS tree.*

Counterexample. Let $V = \{p, x, u, v\}$ and $E = \{p \rightarrow x, x \rightarrow v, p \rightarrow u, u \rightarrow v\}$. This graph is acyclic and contains the path $u \rightarrow v$.

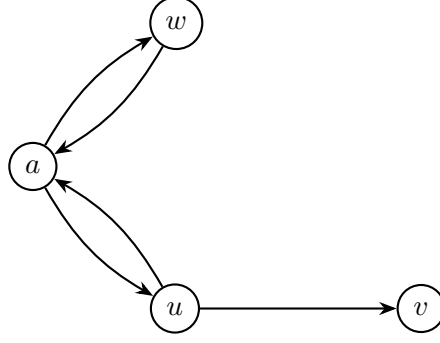


Run DFS from p with adjacency order $p : [x, u]$: tree edges $p \rightarrow x$ and $x \rightarrow v$; v and x finish; then tree edge $p \rightarrow u$, and the edge $u \rightarrow v$ points to a black vertex which is neither an ancestor nor a descendant of u , hence a cross edge. In the resulting DFS tree u is a leaf and v hangs below its sibling x , so u is not an ancestor of v , despite the edge $u \rightarrow v$. $\square$

(c) FALSE.

Claim 1.9. *If u, w are strongly connected, u, v are not strongly connected and $(u, v) \in E$, the inequality $F(w) > F(v)$ on finish times need not hold.*

Counterexample. Let $V = \{a, u, w, v\}$ with $E = \{a \rightarrow w, w \rightarrow a, a \rightarrow u, u \rightarrow a, u \rightarrow v\}$. Then $\{a, u, w\}$ is one strongly connected component, so u and w are strongly connected; v has no outgoing edge, so v forms its own component and u, v are not strongly connected; and $(u, v) \in E$.



Run DFS from a with adjacency orders $a : [w, u]$ and $u : [v, a]$. Then $d[a] = 1$; tree edge $a \rightarrow w$, $d[w] = 2$; back edge $w \rightarrow a$; $F(w) = 3$. Tree edge $a \rightarrow u$, $d[u] = 4$; tree edge $u \rightarrow v$, $d[v] = 5$; $F(v) = 6$. Back edge $u \rightarrow a$; $f[u] = 7$, $f[a] = 8$.

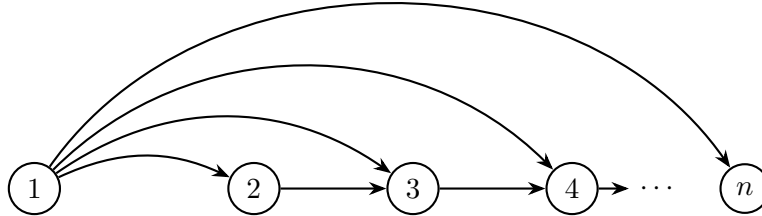
Thus $F(w) = 3 < 6 = F(v)$, contradicting the claimed inequality. $\square$

Remark 1.10. The correct theorem, the one that justifies the SCC algorithms, concerns *maximum* finish times of whole components: if $C \neq C'$ are strongly connected components and some edge goes from C to C' , then $\max_{x \in C} f[x] > \max_{y \in C'} f[y]$. In the example, $\max\{f[a], f[u], f[w]\} = 8 > 6$, as the theorem predicts. The statement in the problem is about an arbitrary vertex w of C instead of the maximizing one, and that is false.

(d) **TRUE.**

Claim 1.11. *There is a graph G on n vertices such that one DFS traversal produces a DFS tree containing a vertex of degree $n - 1$, while another DFS traversal of the same graph produces a DFS tree that is a chain on all n vertices.*

Construction. Let $V = \{1, 2, \dots, n\}$ and $E = \{1 \rightarrow i : 2 \leq i \leq n\} \cup \{i \rightarrow i + 1 : 2 \leq i \leq n - 1\}$.



Traversal A: adjacency order $[2, 3, \dots, n]$ at vertex 1. DFS takes the tree edge $1 \rightarrow 2$, then $2 \rightarrow 3$, then $3 \rightarrow 4$, and so on up to n , since each next vertex is still white. This yields the chain of tree edges $1-2-\dots-n$. When DFS backtracks to vertex 1 and scans the remaining entries $1 \rightarrow 3, \dots, 1 \rightarrow n$, all targets are already finished descendants of 1, so these become forward edges. The DFS tree is a chain on all n vertices.

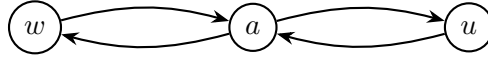
Traversal B: adjacency order $[n, n - 1, \dots, 2]$ at vertex 1. DFS first takes the tree edge $1 \rightarrow n$; vertex n has no outgoing edge, so it is a leaf and finishes. Next it takes $1 \rightarrow (n - 1)$; the only outgoing edge of $n - 1$ is $(n - 1) \rightarrow n$, whose target is black and unrelated to $n - 1$ in the tree, hence a cross edge, so $n - 1$ is a leaf. Inductively, for $i = n - 2, \dots, 2$ the vertex i is discovered directly from 1 and its unique outgoing edge $i \rightarrow i + 1$ points to a vertex already finished, hence is a cross edge. The DFS tree is a star: the root 1 has all of $2, \dots, n$ as children and therefore has degree $n - 1$.

Both traversals run on the same graph and differ only in the adjacency order at vertex 1. $\square$

Problem 4: A Strongly Connected Component Need Not Have a Hamiltonian Cycle

Claim 1.12. *The statement “if $S \subseteq V$ is a strongly connected component of G , then there is a cycle passing through all vertices of S ” is FALSE. Strong connectivity guarantees, for each pair of vertices, a closed walk through both; it does not guarantee a single simple cycle through all vertices of the component.*

Counterexample. Let $V = \{a, u, w\}$ and $E = \{a \rightarrow u, u \rightarrow a, a \rightarrow w, w \rightarrow a\}$.



Every vertex reaches every other through a , so $S = \{a, u, w\}$ is a single strongly connected component.

Suppose some simple cycle C contained all three vertices. In a simple cycle every vertex has exactly one outgoing and one incoming edge of the cycle. The only edges incident to u are $a \rightarrow u$ and $u \rightarrow a$, so C must use both; likewise C must use $a \rightarrow w$ and $w \rightarrow a$. Then a has two outgoing edges in C , which is impossible in a simple cycle. Hence no cycle passes through all of S , and the statement is false. $\square$

Problem 5: Toll-Free Same-State Roads

Statement. $G = (V, E)$ is a weighted directed graph modelling a road network: a vertex is a city, an edge (u, v) is a road, and $w(u, v) > 0$ is its toll. We are given the state $s(u)$ of every city u . The tolls of all roads joining two cities of the same state are removed, i.e. the new weight function is

$$w'(u, v) = \begin{cases} 0, & s(u) = s(v), \\ w(u, v), & s(u) \neq s(v), \end{cases}$$

and we must compute a shortest s - t path with respect to w' . The problem also supplies the fact that the SCC graph of a directed graph is a DAG and can be computed in $O(m + n)$ time.

Step 1: contract the toll-free strongly connected components ($O(m + n)$). Let $E_0 = \{(u, v) \in E : w'(u, v) = 0\}$ be the set of toll-free roads and let $H = (V, E_0)$.

Lemma 1.13. *If x and y lie in the same strongly connected component of H , then $\delta_{w'}(s, x) = \delta_{w'}(s, y)$.*

Proof. There is a walk from x to y inside H and one from y to x , and every edge of H has w' -weight 0, so both walks have w' -weight 0. Appending the $x \rightarrow y$ walk to any s - x walk yields an s - y walk of the same weight, whence $\delta_{w'}(s, y) \leq \delta_{w'}(s, x)$; by symmetry the reverse inequality holds too. $\square$

Compute the strongly connected components $C_1, \dots, C_k$ of H and contract each of them to a single supernode, obtaining $G' = (V', E')$ with $|V'| = k \leq n$ and $|E'| \leq m$; for each ordered pair of distinct supernodes keep one edge of minimum w' -weight among the edges joining them (all this is $O(m + n)$: one pass over E builds H , the SCC algorithm is $O(m + n)$, and the contraction with duplicate removal is $O(m + n)$ using bucketing by tail component). All weights in G' are non-negative, and edges internal to a component are discarded because they all have weight 0.

Lemma 1.14. $\delta_{w'}(s, t) = \delta_{w'}^{G'}(\text{comp}(s), \text{comp}(t))$, and a shortest path in G' expands to a shortest s - t path in G by inserting, inside each supernode, a 0-weight path of H between the entry and the exit city.

Proof. ($\leq$) Take a shortest path in G' and expand it as described; the inserted pieces exist because each component of H is strongly connected, and they have weight 0, so the expanded walk has the same weight, and it is a walk in G , hence $\delta_{w'}(s, t)$ is at most that weight. ($\geq$) Take a shortest s - t path P in G and delete from it every edge whose endpoints lie in the same component of H (such edges have weight 0); the remaining edges, in order, project to a walk from $\text{comp}(s)$ to $\text{comp}(t)$ in G' of weight at most $w'(P)$, because each surviving edge has weight at least the weight of the representative edge kept between the corresponding supernodes. $\square$

Step 2: shortest path in the contracted graph. G' has non-negative real weights, so Dijkstra's algorithm applies. With a Fibonacci heap one run costs $O(|E'| + |V'| \log |V'|) = O(m + n \log n)$.

Algorithm 2: TollFreeShortestPath($G, w, s(\cdot), s, t$)

Input: Directed graph $G = (V, E)$, positive tolls w , state labels $s(\cdot)$, source s , target t .

Output: $\delta_{w'}(s, t)$ and a corresponding path.

- 1 **foreach** $(u, v) \in E$ **do** $w'(u, v) \leftarrow 0$ if $s(u) = s(v)$, else $w(u, v)$;
 - 2 $H \leftarrow (V, E_0)$ where $E_0 = \{(u, v) \in E : w'(u, v) = 0\}$;
 - 3 Compute the SCCs of H and contract them, keeping one minimum-weight edge per ordered pair of distinct supernodes; call the result G' ; // $O(m + n)$
 - 4 **if** $\text{comp}(s) = \text{comp}(t)$ **then return** 0 and a 0-weight path inside that component;
 - 5 Run Dijkstra on G' from $\text{comp}(s)$;
 - 6 **return** $\delta_{w'}^{G'}(\text{comp}(s), \text{comp}(t))$ and the expanded path of Lemma 1.14;
-

What is achieved in $O(m + n)$, and what is not. The following parts of the solution run in $O(m + n)$ time, unconditionally:

- (i) the reweighting $w \mapsto w'$, the construction of H , the computation of its SCCs, and the contraction to G' ;
- (ii) consequently, the decision whether $\delta_{w'}(s, t) = 0$, i.e. whether the journey can be made entirely toll-free: by Lemma 1.13 this holds exactly when $\text{comp}(s) = \text{comp}(t)$ in H ;
- (iii) the whole problem, whenever the contracted graph G' happens to be acyclic — which can be tested in $O(m + n)$ — since one then processes the supernodes in topological order and relaxes their out-edges, exactly as in Problem 1. Acyclicity of G' means precisely that every directed cycle of G uses only toll-free roads.

In general, however, G' contains cycles of positive weight, and the remaining task is genuine single-source shortest paths with arbitrary positive real weights. This residual task is not made easier by the contraction, for the following reason.

Proposition 1.15. *Every instance of the single-source shortest path problem in a directed graph with arbitrary positive real edge weights is an instance of the present problem.*

Proof. Given such a graph, assign every city its own state, i.e. let $s(u) \neq s(v)$ for all $u \neq v$. Then no edge joins two cities of the same state, so $w' = w$, $E_0 = \emptyset$, every SCC of H is a single vertex and $G' = G$. The toll-free reweighting and the contraction do nothing at all, and the instance to be solved after Step 1 is the original shortest path instance. $\square$

Hence an $O(m + n)$ time algorithm for the problem exactly as stated — arbitrary directed road network, arbitrary positive real tolls, arbitrary assignment of states — would in particular be a linear-time algorithm for directed single-source shortest paths with arbitrary positive real weights. Whether such an algorithm exists in the comparison-addition model is a well-known open question; in particular none of the techniques developed in this course produces it: breadth-first search ignores weights, relaxation in topological order requires a DAG, SCC contraction only removes the zero-weight cycles (Proposition 1.15 shows it removes nothing when the states are all distinct), Bellman-Ford costs $O(mn)$, and Dijkstra with a Fibonacci heap costs $O(m + n \log n)$. We are therefore not in a position to claim the bound $O(m + n)$ in general — this is not a proof that no such algorithm exists, but it does show that the requested bound cannot be justified by the structure this problem adds to general shortest paths. We record the honest total running time

$$\underbrace{O(m + n)}_{\text{reweighting, SCCs of } H, \text{ contraction}} + \underbrace{O(m + n \log n)}_{\text{Dijkstra on } G'} = O(m + n \log n),$$

together with the three cases (i)–(iii) above, which are solved in $O(m + n)$. If t is not reachable from s , Dijkstra leaves $\text{comp}(t)$ at ∞ and the algorithm reports that no route exists, which is correct by Lemma 1.14.

Remark 1.16. If one is additionally given that the tolls are integers bounded by a constant C — an assumption that the problem does *not* make — then Dijkstra with a bucket queue (Dial’s algorithm) runs on G' in $O(m + nC) = O(m + n)$ time, and the overall bound becomes $O(m + n)$.

Problem 16: Detecting and Constructing an Eulerian Tour

Task. Given a directed graph $G = (V, E)$ with n vertices and m edges, decide in $O(m + n)$ time whether G is Eulerian, i.e. whether there is a closed tour that traverses every edge exactly once, and print such a tour if one exists.

Call G *balanced* if $\text{indeg}(v) = \text{outdeg}(v)$ for every $v \in V$, and let $V^+ = \{v : \text{indeg}(v) + \text{outdeg}(v) > 0\}$ be the set of vertices incident to at least one edge. The exact characterisation we use is the following; isolated vertices are irrelevant and must not be required to be strongly connected to the rest.

Theorem 1.17. *G has an Euler tour if and only if G is balanced and all vertices of V^+ lie in a single strongly connected component.*

The forward implication is immediate and is proved first; the converse is proved constructively by the algorithm below.

Proof of the “only if” direction. Let T be an Euler tour. Each visit of T to a vertex v uses one incoming and one outgoing edge of v (this also holds at the starting vertex, because T is closed),

and T uses every edge exactly once; hence $\text{indeg}(v) = \text{outdeg}(v)$ for all v . Moreover, if $x, y \in V^+$, then T visits both x and y (each is an endpoint of some edge, and every edge is on T), and since T is a closed walk, following it from x reaches y and vice versa; so x and y are strongly connected. $\square$

Algorithm 3: EulerianTour(G)

Input: Directed graph $G = (V, E)$, $|V| = n$, $|E| = m$.
Output: “Not Eulerian”, or “Eulerian” together with an Euler tour.

```

1 Compute  $\text{indeg}(v)$  and  $\text{outdeg}(v)$  for all  $v$  by one pass over  $E$ ; //  $O(m + n)$ 
2 if  $\text{indeg}(v) \neq \text{outdeg}(v)$  for some  $v$  then return “Not Eulerian”;
3 if  $E = \emptyset$  then return “Eulerian” and the empty tour;
4 Pick  $r$  with  $\text{outdeg}(r) > 0$ ;
5 Run DFS from  $r$  in  $G$  and DFS from  $r$  in the reverse graph  $G^{\text{rev}}$ ;
6 if some vertex of  $V^+$  is unreachable in either search then return “Not Eulerian”;
  //  $G$  is balanced and all edges lie in one SCC: build the tour.
7 For each  $v$  keep a pointer  $\text{next}[v]$  into its adjacency list, initially at its first entry;
8  $C \leftarrow \text{MAXIMALWALK}(r)$ , stored as a doubly linked list of vertex occurrences;
9 Push all occurrences of  $C$  on a stack  $S$ ;
10 while  $S \neq \emptyset$  do
11   pop an occurrence  $p$  of a vertex  $x$  from  $S$ ;
12   advance  $\text{next}[x]$  past edges already used;
13   if  $\text{next}[x]$  points to an unused edge then
14      $W = (x = y_0, y_1, \dots, y_\ell = x) \leftarrow \text{MAXIMALWALK}(x)$ ;
15     insert new occurrences of  $y_1, \dots, y_\ell$  into  $C$  immediately after  $p$ ;
16     push  $p$  and the newly inserted occurrences on  $S$ ;
17 return “Eulerian” and  $C$ ;
```

Algorithm 4: MaximalWalk(x)

```

1  $y \leftarrow x$ ;  $W \leftarrow [x]$ ;
2 while  $\text{next}[y]$  points to an unused edge  $(y, z)$  (after advancing past used ones) do
3   mark  $(y, z)$  used; advance  $\text{next}[y]$ ; append  $z$  to  $W$ ;  $y \leftarrow z$ ;
4 return  $W$ ;
```

Lemma 1.18 (A maximal walk is closed). *Suppose the graph of unused edges is balanced. Then the walk W returned by MAXIMALWALK(x) ends at x .*

Proof. Let W end at y and suppose $y \neq x$. The walk uses edges of the unused graph and never reuses one. Each occurrence of y strictly inside W consumes one incoming and one outgoing edge of y , and the final occurrence consumes one further incoming edge. Hence W consumes one more incoming edge of y than outgoing edges of y . Since $\text{indeg}(y) = \text{outdeg}(y)$ in the unused graph before the walk started, at least one outgoing edge of y is still unused when the walk stops — contradicting the fact that the walk stopped at y only because y has no unused outgoing edge. Hence $y = x$. $\square$

Lemma 1.19 (Splicing point exists). *Let C be a closed walk using an edge set F with $F \neq E$, and assume all vertices of V^+ lie in a single strongly connected component. Then some vertex occurring on C has an unused outgoing edge.*

Proof. Let W be the set of vertices occurring on C and suppose, for contradiction, that every outgoing edge of every vertex of W is used. Let $(a, b) \in E \setminus F$ be an unused edge; then $a \in V^+$ and, since C starts at a vertex $r \in W \cap V^+$, there is a directed path from r to a . This path starts inside W and ends outside W (if $a \in W$ then a itself has the unused outgoing edge (a, b) and we are done), so it contains an edge (p, q) with $p \in W$ and $q \notin W$. By assumption (p, q) is used, i.e. $(p, q) \in F$, so q occurs on C , i.e. $q \in W$ — a contradiction. $\square$

Theorem 1.20. *If G is balanced and all vertices of V^+ lie in a single strongly connected component, then the algorithm outputs an Euler tour of G . Consequently, together with the “only if” direction proved above, the algorithm decides correctly whether G is Eulerian. Its running time is $O(m + n)$.*

Proof. Invariant. Throughout the main loop, C is a closed walk starting and ending at r that uses each of its edges exactly once, and the graph of unused edges is balanced.

Initially, $C = \text{MAXIMALWALK}(r)$; by Lemma 1.18 (applied to the balanced graph G) it is a closed walk at r , and removing the edges of a closed walk from a balanced graph leaves a balanced graph, since a closed walk consumes equally many incoming and outgoing edges at each vertex. So the invariant holds after initialisation.

Iteration. Assume the invariant. If some occurrence p of a vertex x on C has an unused outgoing edge, the algorithm computes $W = (x = y_0, y_1, \dots, y_\ell = x) = \text{MAXIMALWALK}(x)$ in the unused graph, which is balanced; by Lemma 1.18, W is a closed walk at x using at least one previously unused edge. Splicing W into C at p — i.e. inserting occurrences of $y_1, \dots, y_\ell$ immediately after the occurrence p of $y_0 = x$ — again yields a closed walk at r that uses each of its edges exactly once, and the unused graph remains balanced. The invariant is restored, and the number of unused edges has strictly decreased.

Termination and completeness. The loop stops when the stack is empty, i.e. when no occurrence on C has an unused outgoing edge; note that every occurrence ever created is pushed on S at the time it is created, and an occurrence is removed from S only after its vertex is found to have no unused outgoing edge *at that moment*, while an occurrence whose vertex is found to have one is pushed back. Hence at termination no vertex occurring on C has an unused outgoing edge. By Lemma 1.19 this forces $F = E$: all edges are used. By the invariant, C is then a closed walk through every edge exactly once, i.e. an Euler tour.

Complexity. Degree computation and the two DFS traversals cost $O(m + n)$. In the tour construction, each edge is marked used exactly once and each adjacency pointer $\text{next}[v]$ advances only forward, so all calls to MAXIMALWALK together cost $O(m + n)$. The number of occurrences ever created is exactly $1 + |E| = m + 1$ (one at the start, and one per edge used, since a walk with ℓ edges inserts ℓ new occurrences); each is pushed at creation, and an occurrence is pushed again only when a successful splice occurs at it, and there are at most m successful splices (each consumes at least one edge). Hence there are $O(m)$ stack operations, and splicing into a doubly linked list is $O(1)$ per splice. Total: $O(m + n)$. $\square$

2 Dynamic Programming

Problem 1: Weighted Job Scheduling

Counterexample to the “earliest finish time” greedy rule. Let job J_1 have $(s_1, f_1, p_1) = (1, 2, 1)$ and job J_2 have $(s_2, f_2, p_2) = (0, 10, 100)$. Their time spans $[1, 2]$ and $[0, 10]$ overlap. The greedy rule selects J_1 first because $f_1 = 2 < 10 = f_2$, and then J_2 must be rejected; the profit obtained is 1. The optimum schedules J_2 alone and obtains profit 100. Thus the rule, which is optimal when one maximises the *number* of scheduled jobs, is arbitrarily bad in the weighted setting.

Dynamic programming algorithm. Sort the jobs by finish time and relabel them so that $f_1 \leq f_2 \leq \dots \leq f_n$. For each i define

$$\text{pred}(i) = \max\{j < i : f_j \leq s_i\}, \quad \text{pred}(i) = 0 \text{ if no such } j \text{ exists,}$$

the largest index of a job compatible with job i among the jobs that finish no later than i does. Since the finish times are sorted, $\text{pred}(i)$ is obtained by a binary search for s_i in $f_1, \dots, f_{i-1}$, in $O(\log n)$ time per job.

Recurrence. Let $\text{OPT}(i)$ be the maximum profit obtainable from a pairwise compatible subset of $\{J_1, \dots, J_i\}$. Then

$$\text{OPT}(0) = 0, \quad \text{OPT}(i) = \max\left(\underbrace{\text{OPT}(i-1)}_{\text{exclude } J_i}, \underbrace{p_i + \text{OPT}(\text{pred}(i))}_{\text{include } J_i}\right), \quad 1 \leq i \leq n.$$

Algorithm 5: WeightedJobScheduling($J_1, \dots, J_n$)

Input: Jobs with start times s_i , finish times f_i , profits p_i

Output: Maximum profit and an optimal subset of jobs

```

1 Sort the jobs by finish time; relabel so that  $f_1 \leq \dots \leq f_n$ ;
2 foreach  $i = 1, \dots, n$  do compute  $\text{pred}(i)$  by binary search;
3  $\text{OPT}[0] \leftarrow 0$ ;
4 for  $i = 1$  to  $n$  do
5    $\text{OPT}[i] \leftarrow \max(\text{OPT}[i-1], p_i + \text{OPT}[\text{pred}(i)]);$ 
6  $i \leftarrow n$ ;  $A \leftarrow \emptyset$ ;
7 while  $i > 0$  do
8   if  $p_i + \text{OPT}[\text{pred}(i)] \geq \text{OPT}[i-1]$  then
9      $A \leftarrow A \cup \{J_i\}$ ;  $i \leftarrow \text{pred}(i)$ 
10  else
11     $i \leftarrow i-1$ 
12 return  $\text{OPT}[n]$  and  $A$ ;
```

Complexity. Sorting costs $O(n \log n)$, computing all $\text{pred}(i)$ costs $O(n \log n)$, filling the table costs $O(n)$ and the reconstruction costs $O(n)$. Total: $O(n \log n)$, which is polynomial.

Theorem 2.1. For every i , $\text{OPT}(i)$ is the maximum total profit of a pairwise compatible subset of $\{J_1, \dots, J_i\}$.

Proof. Induction on i . For $i = 0$ the only subset is empty and $\text{OPT}(0) = 0$.

Let $i \geq 1$ and assume the claim for all smaller indices. Let O^* be an optimal compatible subset of $\{J_1, \dots, J_i\}$.

- If $J_i \notin O^*$, then O^* is a compatible subset of $\{J_1, \dots, J_{i-1}\}$, so its profit is at most $\text{OPT}(i-1)$ by the induction hypothesis.
- If $J_i \in O^*$, every other job of O^* is compatible with J_i and finishes no later than J_i , hence finishes by time s_i , and therefore has index at most $\text{pred}(i)$ by the definition of $\text{pred}(i)$ and the sorted order. Thus $O^* \setminus \{J_i\}$ is a compatible subset of $\{J_1, \dots, J_{\text{pred}(i)}\}$, whose profit is at most $\text{OPT}(\text{pred}(i))$ by the induction hypothesis; so the profit of O^* is at most $p_i + \text{OPT}(\text{pred}(i))$.

Hence the optimum is at most the maximum of the two quantities. Conversely both quantities are achievable: an optimal subset of $\{J_1, \dots, J_{i-1}\}$ is a compatible subset of $\{J_1, \dots, J_i\}$, and an optimal subset of $\{J_1, \dots, J_{\text{pred}(i)}\}$ together with J_i is compatible, again by the definition of $\text{pred}(i)$. Therefore the optimum equals the recurrence, which is what the algorithm computes. $\square$

Problem 2: Longest Monotonically Increasing Subsequence in $O(n^2)$

Let $L(i)$ be the length of the longest increasing subsequence of $a_1, \dots, a_n$ that ends exactly at index i . Then

$$L(i) = 1 + \max\{L(j) : j < i, a_j < a_i\},$$

where the maximum is 0 if no such j exists. The answer is $\max_{1 \leq i \leq n} L(i)$.

Algorithm 6: LIS($a_1, \dots, a_n$)

```

1 for  $i = 1$  to  $n$  do  $L[i] \leftarrow 1$ ;  $\text{pred}[i] \leftarrow \text{null}$ ;
2 for  $i = 2$  to  $n$  do
3   for  $j = 1$  to  $i - 1$  do
4     if  $a_j < a_i$  and  $L[j] + 1 > L[i]$  then  $L[i] \leftarrow L[j] + 1$ ;  $\text{pred}[i] \leftarrow j$ ;
5  $i^* \leftarrow \arg \max_i L[i]$ ;
6 return  $L[i^*]$  and the subsequence obtained by following  $\text{pred}$  from  $i^*$  and reversing;

```

Complexity. $O(n^2)$ time and $O(n)$ space.

Theorem 2.2. *For every i , the computed value $L[i]$ equals the length of the longest increasing subsequence of A ending exactly at position i , and the reported subsequence is a longest increasing subsequence of A .*

Proof. Induction on i . For $i = 1$ the only such subsequence is a_1 , of length 1.

Let $i \geq 2$ and assume the claim for all $j < i$. An increasing subsequence ending at i either consists of a_i alone, of length 1, or has a second-to-last element a_j with $j < i$ and $a_j < a_i$; deleting a_i from it leaves an increasing subsequence ending at j , of length at most $L(j)$. Conversely, appending a_i to a longest increasing subsequence ending at such a j gives an increasing subsequence ending at i of length $L(j) + 1$. Hence the maximum length is $\max(1, \max_{j < i, a_j < a_i} (L(j) + 1))$, which is exactly the value computed by the inner loop.

Finally, every increasing subsequence ends at some index, so the length of the longest one is $\max_i L(i)$. The pointers pred record, for each i with $L[i] > 1$, an index $j < i$ with $a_j < a_i$ and $L[j] = L[i] - 1$, so following them from i^* produces, in reverse, an increasing subsequence of length $L[i^*]$. $\square$

Problem 3: A Vertex Order Making One Bellman-Ford Pass Exact

Statement. $G = (V, E)$ is a directed graph on n vertices and m edges with real edge weights, and s is the source. We must show that there exists an order of the vertices such that, if the inner **for** loop of the Bellman-Ford algorithm processes the vertices in that order, then after a single iteration of the outer loop $D[v] = \delta(s, v)$ for every v .

Hypothesis needed for the statement to be meaningful. We assume, as in the Bellman-Ford setting, that G has no negative cycle reachable from s . This is not an extra convenience: if a negative cycle is reachable from s and can reach v , then $\delta(s, v) = -\infty$, and no finite label $D[v]$ produced by any finite number of relaxations can store it, so the assertion would be false for every order.

Initialisation and one pass. As in the algorithm, $D[s] = 0$ and $D[v] = \infty$ for $v \neq s$ before the pass. “Processing vertex v ” means performing the relaxations associated with v : either relaxing all out-edges of v , or relaxing all in-edges of v , depending on the formulation used. The order constructed below works for either convention.

Theorem 2.3. *There is an order $v_1, v_2, \dots, v_n$ of V such that after processing the vertices once in this order, $D[v] = \delta(s, v)$ for every $v \in V$.*

Proof. Let R be the set of vertices reachable from s . Since no negative cycle is reachable from s , every $v \in R$ has a shortest s - v path, and it may be taken simple. For each $v \in R$ choose a shortest s - v path P_v with the *minimum number of edges* among all shortest s - v paths, let $e(v)$ be that number of edges, and for $v \neq s$ let $\text{parent}(v) = u$ be the predecessor of v on P_v . Deleting v from P_v leaves a shortest s - u path (optimal substructure), so

$$\delta(s, v) = \delta(s, u) + w(u, v) \quad \text{and} \quad e(u) \leq e(v) - 1 < e(v).$$

Because e strictly decreases along parent pointers, they contain no cycle, and following them from any $v \in R$ reaches s ; hence they form a shortest path tree T rooted at s and spanning R .

Now let $v_1, v_2, \dots, v_{|R|}$ be any ordering of R in which every vertex appears after its parent in T (a preorder or a BFS order of T ; in particular $v_1 = s$, the root), and append the vertices of $V \setminus R$ in arbitrary order at the end.

We prove by induction on i that, at the moment the pass has finished processing v_i , we have $D[v_j] = \delta(s, v_j)$ for all $j \leq i$.

First note that at all times $D[v]$ is either ∞ or the weight of an actual s - v walk (each relaxation extends such a walk by one edge), so $D[v] \geq \delta(s, v)$ always; it therefore suffices to establish the upper bounds.

Base. $D[v_1] = D[s] = 0 = \delta(s, s)$ by initialisation. (Note $\delta(s, s) = 0$ since no negative cycle passes through s .)

Step. Let $i \geq 2$ and let $u = \text{parent}(v_i) = v_j$ with $j < i$. By the induction hypothesis $D[u] = \delta(s, u)$ once v_j has been processed. The edge (u, v_i) is relaxed while v_j is processed (out-edge convention) or while v_i is processed (in-edge convention); in either case this happens after $D[u]$ has attained the value $\delta(s, u)$ and no later than the processing of v_i . That relaxation yields

$$D[v_i] \leq D[u] + w(u, v_i) = \delta(s, u) + w(u, v_i) = \delta(s, v_i),$$

the last equality because (u, v_i) is a tree edge of a shortest path tree. Together with $D[v_i] \geq \delta(s, v_i)$ this gives $D[v_i] = \delta(s, v_i)$. Since labels never increase, the values of $v_1, \dots, v_{i-1}$ remain correct.

Finally, for $v \in V \setminus R$ we have $\delta(s, v) = \infty$, and $D[v]$ stays ∞ : a relaxation could lower $D[v]$ only through an edge from a vertex with a finite label, i.e. from a vertex reachable from s , which would make v reachable from s . Hence $D[v] = \delta(s, v)$ for all $v \in V$ after one pass. $\square$

Remark 2.4 (Acyclicity is sufficient but not necessary). If G is a DAG, one may take the order to be any topological order of G , and the same induction (every in-edge of a vertex is relaxed before the vertex is processed) shows that a single pass suffices — for *every* real weight function simultaneously, and without computing any shortest paths first.

Acyclicity is, however, not necessary, as the theorem above already shows: a valid order exists for *every* graph without a negative cycle reachable from s , cyclic or not. For a concrete cyclic example, let $V = \{s, a, b\}$ with edges $s \rightarrow a$ of weight 1, $s \rightarrow b$ of weight 10, $a \rightarrow b$ of weight 1 and $b \rightarrow a$ of weight 1. The graph contains the cycle $a \rightarrow b \rightarrow a$, yet after one pass in the order s, a, b we get $D[a] = 1 = \delta(s, a)$ and $D[b] = \min(10, D[a] + 1) = 2 = \delta(s, b)$, both correct. We therefore do *not* claim any equivalence between the existence of a valid one-pass order and acyclicity.

Remark 2.5 (On the position of s). In the order constructed above, s precedes every vertex reachable from s , since it is the root of the shortest path tree. Vertices not reachable from s may be placed anywhere, because their labels remain ∞ throughout, which is the correct value. Thus we do not need to assume that every vertex is reachable from s , nor that s is globally first.

Problem 4: Stacking 3-D Boxes for Maximum Height

Setup. There are n types of boxes, box i having height $h(i)$, width $w(i)$ and depth $d(i)$. A box may be rotated so that any of its three sides is the base, and any number of copies of a type may be used. A box B may be placed on top of a box B' if and only if both base dimensions of B are strictly smaller than the corresponding base dimensions of B' .

Step 1: generate the rotations. For each type, generate the three rotations obtained by choosing one of its three dimensions as the height; for each rotation the remaining two dimensions form the base, and we store it normalised as (w, d) with $w \geq d$ (rotating a box about its vertical axis is free, so a base $\{x, y\}$ may always be presented with its larger side first). This yields a list of $3n$ box instances, each with a height and a normalised base.

Lemma 2.6 (Normalisation, and why unlimited copies cost nothing).

- (i) An instance with normalised base (w_i, d_i) may be placed on an instance with normalised base (w_j, d_j) if and only if $w_j > w_i$ and $d_j > d_i$.
- (ii) In any valid stack the two base dimensions strictly decrease from bottom to top, so no instance occurs twice in a stack. Consequently the valid stacks are exactly the sequences of pairwise distinct instances $B_{i_1}, \dots, B_{i_r}$ (bottom to top) that satisfy the condition of (i) at each interface, and allowing unlimited copies of a box type adds nothing beyond the $3n$ instances already generated.

Proof. (i) Because a box may be turned about its vertical axis at no cost, the upper box fits if *some* matching of its two base dimensions with the two base dimensions of the lower box is strictly increasing. Write the upper base as $a_1 \geq a_2$ and the lower as $b_1 \geq b_2$. If $a_1 < b_1$ and $a_2 < b_2$, the identity matching works. Conversely, suppose some matching works. If it matches $a_1 < b_1$ and $a_2 < b_2$ we are done; otherwise it matches $a_1 < b_2$ and $a_2 < b_1$, and then $a_1 < b_2 \leq b_1$ and $a_2 \leq a_1 < b_2$, so again $a_1 < b_1$ and $a_2 < b_2$.

(ii) By (i), the pair (w, d) decreases strictly in both coordinates at every interface, hence strictly along the whole stack, so all instances of a stack are pairwise distinct. A box type may still be used several times in one stack, but only with different rotations, i.e. as different instances of the $3n$ generated ones; the same rotation of the same type can never appear twice, since two such instances have identical bases. Therefore restricting attention to stacks of pairwise distinct generated instances loses no valid stack. $\square$

Step 2: sort. Sort the $3n$ instances by *decreasing* w , breaking ties by decreasing d . Let the sorted list be $B_1, \dots, B_{3n}$ with heights h_i and bases (w_i, d_i) . If instance B_j can support instance B_i , then $w_j > w_i$, hence $j < i$: every admissible supporting instance occurs strictly earlier in the sorted order.

Step 3: the recurrence. Let $H(i)$ be the maximum total height of a stack whose **topmost** instance is B_i . Since the instance directly below B_i , if any, must be some B_j with $w_j > w_i$ and $d_j > d_i$, and since all such j satisfy $j < i$,

$$H(i) = h_i + \max\{H(j) : j < i, w_j > w_i, d_j > d_i\},$$

with the convention that the maximum is 0 when no such j exists. The answer is $\max_i H(i)$.

Algorithm 7: BoxStacking(h, w, d)

```

1 Generate the  $3n$  rotations, normalising each base so that  $w \geq d$ ;
2 Sort the instances by decreasing  $w$ , ties broken by decreasing  $d$ ;
3 for  $i = 1$  to  $3n$  do  $H[i] \leftarrow h_i$ ; below[ $i$ ]  $\leftarrow$  null;
4 for  $i = 1$  to  $3n$  do
5   for  $j = 1$  to  $i - 1$  do
6     if  $w_j > w_i$  and  $d_j > d_i$  and  $H[j] + h_i > H[i]$  then  $H[i] \leftarrow H[j] + h_i$ ; below[ $i$ ]  $\leftarrow j$ ;
7 return  $\max_i H[i]$  and the stack reconstructed through below;
```

Complexity. Generating the rotations is $O(n)$, sorting is $O(n \log n)$, and the double loop over $3n$ instances is $O(n^2)$. Total: $O(n^2)$ time and $O(n)$ space.

Theorem 2.7. *For every i , the computed value $H[i]$ equals the maximum height of a valid stack whose topmost instance is B_i ; consequently $\max_i H[i]$ is the maximum height of any valid stack.*

Proof. Induction on i in the sorted order.

Base. For $i = 1$ there is no $j < 1$, so the algorithm returns h_1 . Indeed no instance can support B_1 : a supporter would need a strictly larger first base dimension and would therefore precede B_1 in the sorted order, which is impossible. Hence the only stack with B_1 on top is B_1 alone.

Step. Let $i \geq 2$ and assume the claim for all $j < i$. Consider any valid stack S with B_i on top. If B_i is the only instance of S , its height is h_i . Otherwise let B_j be the instance immediately below B_i ; validity gives $w_j > w_i$ and $d_j > d_i$ by Lemma 2.6(i), hence $j < i$. The part of S from B_j downwards is itself a valid stack with B_j on top, so its height is at most $H(j)$ by the induction hypothesis, and the height of S is at most $h_i + H(j)$. Conversely, for any $j < i$ with $w_j > w_i$ and $d_j > d_i$, a maximum-height stack with B_j on top may be extended by placing B_i on it, which is valid, and B_i does not already occur in that stack because its base is strictly smaller than the base of its top instance B_j ; this gives a stack of height $h_i + H(j)$ with B_i on top. (By Lemma 2.6(ii) the permission to use several copies of a type is fully accounted for by the $3n$ instances.) Therefore

the maximum height of a stack with B_i on top equals $h_i + \max\{H(j) : j < i, w_j > w_i, d_j > d_i\}$, exactly the value computed.

Finally, the topmost instance of an optimal stack is some B_i , so the maximum over i of $H(i)$ is the optimal height. $\square$

Problem 5: Maximum Number of Non-Crossing Bridges

Setup. The southern cities have x -coordinates $a_1, \dots, a_n$ and the northern cities have x -coordinates $b_1, \dots, b_n$; no order is assumed among the a_i , nor among the b_i . A bridge may join southern city i only to northern city i , so the candidate bridges are the n segments β_i joining the point $(a_i, \text{south bank})$ to the point $(b_i, \text{north bank})$. Distinct cities on the same bank occupy distinct positions, so the a_i are pairwise distinct and so are the b_i . We must compute the maximum number of pairwise non-crossing bridges.

Claim 2.8. *Let $i \neq j$ with $a_i < a_j$. The bridges β_i and β_j do not cross if and only if $b_i < b_j$.*

Proof. Put the southern bank on the line $y = 0$ and the northern bank on the line $y = 1$, so that bridge β_k is the segment from $(a_k, 0)$ to $(b_k, 1)$, i.e. the set of points $(x_k(t), t)$ for $t \in [0, 1]$ with $x_k(t) = (1 - t)a_k + tb_k$. Two bridges β_i, β_j meet if and only if $x_i(t) = x_j(t)$ for some $t \in [0, 1]$, i.e. if and only if the affine function $f(t) = x_j(t) - x_i(t)$ has a zero in $[0, 1]$. Here $f(0) = a_j - a_i > 0$ and $f(1) = b_j - b_i \neq 0$, since the coordinates on each bank are distinct.

If $b_i < b_j$ then $f(1) > 0$; an affine function that is positive at both endpoints of $[0, 1]$ is positive on all of $[0, 1]$, so f has no zero and the bridges do not cross. If $b_j < b_i$ then $f(1) < 0$, and since $f(0) > 0$ the intermediate value theorem gives a zero $t^* \in (0, 1)$, so the bridges cross. $\square$

Reduction to the longest increasing subsequence. Sort the indices by increasing southern coordinate, obtaining a permutation σ with $a_{\sigma(1)} < a_{\sigma(2)} < \dots < a_{\sigma(n)}$, and define the sequence

$$c_k = b_{\sigma(k)}, \quad k = 1, \dots, n.$$

A set S of bridges is pairwise non-crossing if and only if, by Claim 2.8, the map $a \mapsto b$ restricted to S is increasing, i.e. if and only if the indices of S , listed in σ -order as $k_1 < k_2 < \dots < k_r$, satisfy $c_{k_1} < c_{k_2} < \dots < c_{k_r}$. Thus the sets of pairwise non-crossing bridges correspond exactly to the increasing subsequences of $c_1, \dots, c_n$, and the maximum number of non-crossing bridges is precisely the length of a longest increasing subsequence of $c_1, \dots, c_n$.

Algorithm 8: $\text{MaxNonCrossingBridges}(a_1, \dots, a_n, b_1, \dots, b_n)$

- 1 Sort the indices $1, \dots, n$ by increasing a_i , obtaining σ ;
 - 2 $c_k \leftarrow b_{\sigma(k)}$ for $k = 1, \dots, n$;
 - 3 **return** $\text{LIS}(c_1, \dots, c_n)$, computed by the algorithm of Problem 2, together with the corresponding set of bridges;
-

Complexity. $O(n \log n)$ for sorting and $O(n^2)$ for the LIS computation of Problem 2, i.e. $O(n^2)$ overall, which is polynomial. (With the $O(n \log n)$ patience-sorting variant of LIS the total becomes $O(n \log n)$.)

Problem 6: Edit Distance

Let $A[1..n]$ and $B[1..m]$ be the two strings and let $E[i][j]$ denote the edit distance between the prefixes $A[1..i]$ and $B[1..j]$, where the allowed operations are deleting a character of A , inserting a character into A , and changing a character of A , each of cost 1.

$$E[i][0] = i \quad (0 \leq i \leq n), \quad E[0][j] = j \quad (0 \leq j \leq m),$$

$$E[i][j] = \begin{cases} E[i-1][j-1], & A[i] = B[j], \\ 1 + \min(E[i-1][j], E[i][j-1], E[i-1][j-1]), & A[i] \neq B[j], \end{cases}$$

the three terms in the second case corresponding to deleting $A[i]$, inserting $B[j]$, and changing $A[i]$ into $B[j]$.

Algorithm 9: EditDistance(A, B)

```

1 for  $i = 0$  to  $n$  do  $E[i][0] \leftarrow i$ ;
2 for  $j = 0$  to  $m$  do  $E[0][j] \leftarrow j$ ;
3 for  $i = 1$  to  $n$  do
4   for  $j = 1$  to  $m$  do
5     if  $A[i] = B[j]$  then
6        $E[i][j] \leftarrow E[i-1][j-1]$ 
7     else
8        $E[i][j] \leftarrow 1 + \min(E[i-1][j], E[i][j-1], E[i-1][j-1])$ 
9 return  $E[n][m]$ ;
```

Complexity. $O(nm)$ time and $O(nm)$ space; the space can be reduced to $O(\min(n, m))$ if only the distance is needed, by keeping two rows.

Theorem 2.9. $E[i][j]$ equals the edit distance between $A[1..i]$ and $B[1..j]$ for all $0 \leq i \leq n$, $0 \leq j \leq m$.

Proof. Induction on $i + j$. The base cases are $E[i][0] = i$ (the only way to turn $A[1..i]$ into the empty string is to delete its i characters, and i deletions suffice) and $E[0][j] = j$ (symmetrically, j insertions).

Let $i, j \geq 1$ and assume the claim for all pairs with a smaller sum. We use the standard reformulation of edit distance in terms of *alignments*. An alignment of $A[1..i]$ with $B[1..j]$ is a set M of pairs (p, q) with $1 \leq p \leq i$, $1 \leq q \leq j$, in which every index of A and every index of B occurs at most once and which is monotone (if $(p, q), (p', q') \in M$ and $p < p'$ then $q < q'$); its cost is the number of *mismatched* pairs $(p, q) \in M$ (those with $A[p] \neq B[q]$) plus the number of indices of A and of B left unpaired.

- Any alignment of cost c can be realised by c operations: substitute at each mismatched pair, delete each unpaired character of A , insert each unpaired character of B . Hence the edit distance is at most the minimum alignment cost.
- Conversely, from any sequence of c operations transforming $A[1..i]$ into $B[1..j]$ we obtain an alignment of cost at most c : follow each character of A through the sequence, and pair p with q when the character originally at position p of A survives (possibly after changes) as the character at position q of B . This set of pairs is monotone because the operations never reorder characters; a pair is mismatched only if the character was changed at least once, an unpaired index of A corresponds to a deleted character, and an unpaired index of B to an

inserted one, so each unit of cost of the alignment is charged to a distinct operation. Hence the minimum alignment cost is at most the edit distance.

So the edit distance equals the minimum alignment cost. Now consider an optimal alignment M of $A[1..i]$ with $B[1..j]$ and look at the indices i and j . By monotonicity, the only pair of M that can involve i or j as its largest entries is (i, j) itself, so exactly one of the following holds.

- i is unpaired (deletion): the rest of M is an alignment of $A[1..i-1]$ with $B[1..j]$, and the cost is $1 + (\text{cost of that alignment}) \geq 1 + E[i-1][j]$ by the induction hypothesis; conversely this value is attainable.
- j is unpaired (insertion): symmetrically the cost is $1 + E[i][j-1]$.
- $(i, j) \in M$: the rest of M aligns $A[1..i-1]$ with $B[1..j-1]$, and the cost is $[A[i] \neq B[j]] + E[i-1][j-1]$.

Note that if i is paired with some $q < j$ then j is unpaired, which is the second case, and symmetrically; so the three cases are exhaustive. Hence the edit distance equals $\min(1 + E[i-1][j], 1 + E[i][j-1], [A[i] \neq B[j]] + E[i-1][j-1])$. When $A[i] \neq B[j]$ this is exactly the second line of the recurrence. When $A[i] = B[j]$ the third option costs $E[i-1][j-1]$, and it is never worse than the other two, because $E[i-1][j-1] \leq E[i-1][j] + 1$ and $E[i-1][j-1] \leq E[i][j-1] + 1$ (adjacent entries of the table differ by at most 1, since one extra character can always be handled by one extra insertion or deletion); so the minimum equals $E[i-1][j-1]$, which is the first line of the recurrence. In both cases the recurrence computes the edit distance, completing the induction. $\square$

Problem 7: Matrix Chain Multiplication

Let $M_1, \dots, M_n$ have dimensions $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, so that M_i is $p_{i-1} \times p_i$. Let $m[i][j]$ be the minimum number of scalar multiplications needed to compute $M_i M_{i+1} \dots M_j$. Then

$$m[i][i] = 0 \quad (1 \leq i \leq n), \quad m[i][j] = \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p_{i-1} p_k p_j) \quad (i < j).$$

Justification of the recurrence (following the hint). Fix an optimal parenthesisation of $M_i \dots M_j$ with $i < j$ and look at the *last* multiplication it performs. At that moment only two matrices are left: the product of a prefix $M_i \dots M_k$, which is a $p_{i-1} \times p_k$ matrix, and the product of the suffix $M_{k+1} \dots M_j$, which is a $p_k \times p_j$ matrix, for some split point k with $i \leq k < j$. This last multiplication costs exactly $p_{i-1} p_k p_j$ scalar multiplications. The two sub-products must themselves be computed by optimal parenthesisations: replacing the parenthesisation of a sub-product by a cheaper one leaves the rest of the computation unchanged and strictly reduces the total cost, contradicting optimality. Hence the optimal cost equals $m[i][k] + m[k+1][j] + p_{i-1} p_k p_j$ for that particular k ; since k is not known in advance, we minimise over all $i \leq k < j$.

Algorithm 10: MatrixChainOrder($p_0, \dots, p_n$)

```

1 for  $i = 1$  to  $n$  do  $m[i][i] \leftarrow 0$ ;
2 for  $\ell = 2$  to  $n$  do
3   for  $i = 1$  to  $n - \ell + 1$  do
4      $j \leftarrow i + \ell - 1$ ;  $m[i][j] \leftarrow \infty$ ;
5     for  $k = i$  to  $j - 1$  do
6        $q \leftarrow m[i][k] + m[k+1][j] + p_{i-1} p_k p_j$ ;
7       if  $q < m[i][j]$  then  $m[i][j] \leftarrow q$ ;  $s[i][j] \leftarrow k$ ;
8 return  $m[1][n]$  and the optimal parenthesisation recovered from  $s$ ;
```

Iteration order. The table is filled by increasing chain length $\ell = j - i + 1$, because computing $m[i][j]$ uses $m[i][k]$ and $m[k+1][j]$, whose chain lengths $k - i + 1$ and $j - k$ are both strictly smaller than ℓ .

Complexity. There are $O(n^2)$ entries and each requires a minimisation over at most n split points, so the running time is $O(n^3)$ and the space is $O(n^2)$.

Theorem 2.10. *For all $1 \leq i \leq j \leq n$, the computed $m[i][j]$ is the minimum number of scalar multiplications needed to compute $M_i \cdots M_j$.*

Proof. Strong induction on the chain length $\ell = j - i + 1$. For $\ell = 1$ no multiplication is needed and $m[i][i] = 0$. For $\ell \geq 2$, assume correctness for all chains of length smaller than ℓ . By the argument above, the true optimum for $M_i \cdots M_j$ equals $\min_{i \leq k < j} (\text{opt}(i, k) + \text{opt}(k+1, j) + p_{i-1}p_kp_j)$; both sub-chains have length smaller than ℓ , so by the induction hypothesis $\text{opt}(i, k) = m[i][k]$ and $\text{opt}(k+1, j) = m[k+1][j]$ have already been computed correctly when the entry (i, j) is filled. Hence the minimisation performed by the algorithm yields the optimum. $\square$

3 Shortest Paths

Problem 1: Feasible and Tight Labels

Statement. $G = (V, E)$ is a directed graph with real edge weights and no negative cycle, $s \in V$, and $L : V \rightarrow \mathbb{R}$ satisfies

- (a) $L(s) = 0$;
- (b) for each $v \in V \setminus \{s\}$: $L(v) \leq L(u) + w(u, v)$ for *every* edge $(u, v) \in E$, and $L(v) = L(u) + w(u, v)$ for *at least one* edge $(u, v) \in E$.

The problem asks to prove that $L(v) = \delta(s, v)$ for every v . One half of this is true unconditionally; the statement as a whole is false, and we give a counterexample and the minimal repair.

Lemma 3.1 (Lower bound; no extra hypothesis). *Under (a) and (b), $L(v) \leq \delta(s, v)$ for every $v \in V$, with the convention $\delta(s, v) = +\infty$ if v is not reachable from s .*

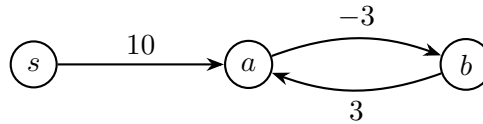
Proof. If v is unreachable the inequality is trivial because $L(v)$ is a real number. Otherwise let $P : s = x_0, x_1, \dots, x_k = v$ be any s - v path. Applying the inequality of (b) to each edge of P and summing telescopically,

$$L(v) = L(x_k) \leq L(x_0) + \sum_{i=1}^k w(x_{i-1}, x_i) = 0 + w(P).$$

Taking the minimum over all s - v paths P gives $L(v) \leq \delta(s, v)$. □

Proposition 3.2 (The statement as posed is false). *There is a directed graph with real edge weights and no negative cycle, and a map L satisfying (a) and (b), such that $L \neq \delta(s, \cdot)$.*

Counterexample. Let $V = \{s, a, b\}$ with edges $s \rightarrow a$ of weight 10, $a \rightarrow b$ of weight -3 and $b \rightarrow a$ of weight 3.



The only cycle is $a \rightarrow b \rightarrow a$, of weight 0, so there is no negative cycle. The true distances are $\delta(s, a) = 10$ and $\delta(s, b) = 7$.

Define $L(s) = 0$, $L(a) = 5$, $L(b) = 2$. We verify (a) and (b):

- $L(s) = 0$.
- In-edges of a : $L(a) = 5 \leq L(s) + 10 = 10$ and $L(a) = 5 = L(b) + 3 = 2 + 3$. So all inequalities hold and the edge (b, a) is tight.
- In-edge of b : $L(b) = 2 = L(a) + (-3) = 5 - 3$, so the inequality holds and the edge (a, b) is tight.

All the hypotheses are satisfied, yet $L(a) = 5 \neq 10 = \delta(s, a)$ and $L(b) = 2 \neq 7 = \delta(s, b)$. The reason is that the tight edges chosen for a and for b form a closed chain of weight 0 which is never anchored at s . □

Theorem 3.3 (Sharp corrected statement). *Assume (a) and (b), and for each $v \neq s$ fix one tight in-edge, i.e. a vertex $\pi(v)$ with $L(v) = L(\pi(v)) + w(\pi(v), v)$. Let $v \in V$ and consider the chain $v, \pi(v), \pi(\pi(v)), \dots$*

- (i) *If the chain reaches s , then $L(v) = \delta(s, v)$ and v is reachable from s .*

(ii) Otherwise the chain repeats a vertex, and the corresponding cycle of tight edges has total weight exactly 0.

Proof. (i) Suppose the chain reaches s : $v = u_0$, $u_1 = \pi(u_0)$, $\dots$, $u_k = s$. Reversing it gives an actual directed path $P : s = u_k, u_{k-1}, \dots, u_0 = v$ in G (each $\pi(u_i) \rightarrow u_i$ is an edge). Summing the tight equalities $L(u_i) = L(u_{i+1}) + w(u_{i+1}, u_i)$ for $i = k-1, \dots, 0$ telescopes to

$$L(v) = L(s) + \sum_{i=0}^{k-1} w(u_{i+1}, u_i) = w(P) \geq \delta(s, v).$$

With Lemma 3.1 this gives $L(v) = \delta(s, v)$, and P witnesses reachability.

(ii) If the chain never reaches s , it stays inside the finite set $V \setminus \{s\}$ forever and therefore repeats a vertex, say $u_p = u_q$ with $p < q$. The edges (u_{i+1}, u_i) for $p \leq i < q$ form a directed cycle C of G , all of whose edges are tight, and summing the equalities $L(u_i) = L(u_{i+1}) + w(u_{i+1}, u_i)$ around C telescopes to $0 = w(C)$. $\square$

Corollary 3.4 (Minimal repair of the statement). *If, in addition to the hypotheses of the problem, G has no cycle of weight 0 — equivalently, given that G has no negative cycle, if every cycle of G has strictly positive weight — then $L(v) = \delta(s, v)$ for every $v \in V$, and in particular every vertex is reachable from s .*

Proof. Case (ii) of Theorem 3.3 would produce a cycle of weight 0, which is excluded; hence for every v the chain reaches s and case (i) applies. $\square$

Remark 3.5. No other hypothesis is needed: reachability of all vertices from s is a *consequence* of the corrected hypothesis, not an extra assumption, and the existence of a vertex with no in-edge other than s is already excluded by (b), which requires a tight in-edge at every $v \neq s$. Proposition 3.2 shows that some such additional hypothesis is necessary: with a zero-weight cycle present, the conclusion can fail.

Problem 2: In-Place (Single-Array) Bellman-Ford

The algorithm in question. A single array L is used, and updates are written into it immediately:

Algorithm 11: InPlaceBellmanFord(G, s)

```

1  $L(s) \leftarrow 0$ ;
2 foreach  $v \in V \setminus \{s\}$  do  $L(v) \leftarrow \infty$ ;
3 for  $i = 1$  to  $n - 1$  do
4   foreach  $(u, v) \in E$  in a fixed order do
5     if  $L(u) + w(u, v) < L(v)$  then  $L(v) \leftarrow L(u) + w(u, v)$ ;
6 return  $L$ ;
```

Answer. Yes: at the end, $L(v) = \delta(s, v)$ for every v , provided G has no negative cycle. We prove it by sandwiching L between two bounds. Throughout, $L^{(i)}(v)$ denotes the value of $L(v)$ at the end of pass i (so $L^{(0)}$ is the initialisation), and $\delta_i(s, v)$ denotes the minimum weight of an s - v walk using at most i edges, which is the value computed by the standard double-buffered Bellman-Ford after i passes.

Lemma 3.6 (Every label is realised by a walk). *At every moment of the execution, if $L(v) < \infty$ then $L(v)$ is the weight of some s - v walk in G . Consequently $L(v) \geq \delta(s, v)$ at all times.*

Proof. Induction on the number of updates performed. Initially $L(s) = 0$ is the weight of the empty walk and every other label is ∞ . An update sets $L(v) \leftarrow L(u) + w(u, v)$ with $L(u) < \infty$; by the induction hypothesis $L(u)$ is the weight of an s - u walk W , so the new value is the weight of the s - v walk W followed by the edge (u, v) . Since G has no negative cycle, the minimum weight of an s - v walk equals $\delta(s, v)$, so every realised value is at least $\delta(s, v)$. $\square$

Lemma 3.7 (The in-place labels dominate the synchronous ones). *For every $i \geq 0$ and every $v \in V$: $L^{(i)}(v) \leq \delta_i(s, v)$.*

Proof. Induction on i .

Base $i = 0$. $L^{(0)}(s) = 0 = \delta_0(s, s)$ and $L^{(0)}(v) = \infty = \delta_0(s, v)$ for $v \neq s$.

Step. Assume $L^{(i-1)}(u) \leq \delta_{i-1}(s, u)$ for all u . Three observations about pass i :

- (1) Labels never increase, so at *every* moment during pass i the current value of any label $L(u)$ satisfies $L(u) \leq L^{(i-1)}(u)$. (The labels at the beginning of pass i are exactly $L^{(i-1)}$.)
- (2) Every edge (u, v) is scanned exactly once during pass i . At that moment the algorithm makes $L(v)$ at most the candidate $L(u) + w(u, v)$, where $L(u)$ is the *current* in-place value of u ; by (1) and the induction hypothesis,

$$L(u) + w(u, v) \leq L^{(i-1)}(u) + w(u, v) \leq \delta_{i-1}(s, u) + w(u, v).$$

Hence, at the end of pass i , $L^{(i)}(v) \leq \delta_{i-1}(s, u) + w(u, v)$, again because labels never increase after that moment.

- (3) Also $L^{(i)}(v) \leq L^{(i-1)}(v) \leq \delta_{i-1}(s, v)$.

Combining (2) over all in-edges of v with (3),

$$L^{(i)}(v) \leq \min\left(\delta_{i-1}(s, v), \min_{(u,v) \in E} (\delta_{i-1}(s, u) + w(u, v))\right) = \delta_i(s, v),$$

the last equality being the Bellman-Ford recurrence: a minimum-weight s - v walk with at most i edges either uses at most $i - 1$ edges, or ends with an edge (u, v) preceded by a minimum-weight s - u walk with at most $i - 1$ edges. $\square$

Note that the argument nowhere claims that the two algorithms hold the same labels; it only uses that the in-place labels are never larger than the synchronous ones at the corresponding moments, and this is independent of the order in which the edges are scanned inside a pass.

Theorem 3.8. *If G has no negative cycle, then after the $n - 1$ passes of the in-place algorithm $L(v) = \delta(s, v)$ for every $v \in V$ (including $L(v) = \infty$ for v unreachable from s).*

Proof. By Lemma 3.6, $L^{(n-1)}(v) \geq \delta(s, v)$. Since there is no negative cycle, some shortest s - v path is simple and therefore has at most $n - 1$ edges, so $\delta(s, v) = \delta_{n-1}(s, v)$; Lemma 3.7 with $i = n - 1$ gives $L^{(n-1)}(v) \leq \delta_{n-1}(s, v) = \delta(s, v)$. Hence equality. If v is unreachable, no label of v is ever finite, because an update of $L(v)$ requires a finite $L(u)$ on an edge (u, v) , and by Lemma 3.6 a finite label means a walk from s ; so $L(v) = \infty = \delta(s, v)$. $\square$

Problem 3: Augmenting Bellman-Ford to Report Shortest Paths

Task. Augment Bellman-Ford with a data structure of size $O(n)$ from which, for any v , a shortest s - v path can be reported in time proportional to the number of its edges.

Algorithm 12: BellmanFordWithPaths(G, s)

```

1  $L[s] \leftarrow 0; \pi[s] \leftarrow \text{null};$ 
2 foreach  $v \neq s$  do  $L[v] \leftarrow \infty; \pi[v] \leftarrow \text{null};$ 
3 for  $i = 1$  to  $n - 1$  do
4   foreach  $(u, v) \in E$  do
5     if  $L[u] + w(u, v) < L[v]$  then
6        $L[v] \leftarrow L[u] + w(u, v);$ 
7        $\pi[v] \leftarrow u;$ 
8 return  $L[\cdot]$  and  $\pi[\cdot];$ 
```

Algorithm 13: ReportPath(v)

```

1 if  $L[v] = \infty$  then return “ $v$  is not reachable from  $s$ ”;
2  $P \leftarrow [v];$ 
3 while  $\pi[v] \neq \text{null}$  do  $v \leftarrow \pi[v];$  prepend  $v$  to  $P;$ 
4 return  $P;$ 
```

Storage. One extra array π of n entries, i.e. $O(n)$ space, and no change to the $O(mn)$ running time of Bellman-Ford.

Throughout this subsection G has no negative cycle. We first isolate the invariant that the predecessor array really satisfies. It is *not* true that the recorded edge stays tight at all times: if $L[v]$ was set using (u, v) and $L[u]$ later decreases while (u, v) is not scanned again, then $L[v] > L[u] + w(u, v)$ at that moment. The correct invariant is an inequality, and it is enough.

Lemma 3.9 (Predecessor invariant). *At every moment of the execution, for every v with $\pi[v] \neq \text{null}$,*

$$L[v] \geq L[\pi[v]] + w(\pi[v], v),$$

and $L[\pi[v]] < \infty$.

Proof. Consider the moment at which $\pi[v]$ receives its current value u . At that instant $L[v]$ is set to $L[u] + w(u, v)$, so the inequality holds with equality and $L[u] < \infty$. Afterwards, and until $\pi[v]$ changes again, $L[v]$ is not modified (any modification of $L[v]$ is accompanied by a new assignment to $\pi[v]$), while $L[u]$ can only decrease. Hence the right-hand side can only decrease and the inequality is preserved. $\square$

Lemma 3.10 (The predecessor graph is acyclic). *At every moment, the graph $G_\pi = \{(\pi[v], v) : v \in V, \pi[v] \neq \text{null}\}$ contains no directed cycle.*

Proof. Every vertex has at most one entry in π , so a cycle in G_π , if it ever appears, appears at the moment of a single assignment. Suppose that an assignment $\pi[v_0] \leftarrow v_{k-1}$ creates the cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k-1} \rightarrow v_0$, meaning that $\pi[v_j] = v_{j-1}$ for $j = 1, \dots, k-1$ already holds just before the assignment. All the labels $L[v_0], \dots, L[v_{k-1}]$ are finite just before the assignment: $L[v_{k-1}]$ is finite because the update is triggered, and each $L[v_j]$ with $j \leq k-1$ is finite because $\pi[v_j] \neq \text{null}$ was set earlier; in particular $L[v_0]$ is finite because $\pi[v_1] = v_0$ was set earlier, which required a finite $L[v_0]$.

Just before the assignment, Lemma 3.9 gives

$$L[v_j] \geq L[v_{j-1}] + w(v_{j-1}, v_j) \quad (j = 1, \dots, k-1),$$

and the relaxation condition that triggers the assignment gives the *strict* inequality

$$L[v_0] > L[v_{k-1}] + w(v_{k-1}, v_0).$$

Adding all k relations, the sums $\sum_j L[v_j]$ on the two sides are equal and finite, so they cancel and we obtain

$$0 > \sum_{j=1}^{k-1} w(v_{j-1}, v_j) + w(v_{k-1}, v_0),$$

i.e. the cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k-1} \rightarrow v_0$ has negative weight — contradicting the hypothesis that G has no negative cycle. Hence no cycle is ever created. $\square$

Theorem 3.11. *Assume G has no negative cycle. At termination:*

- (i) $L[v] = \delta(s, v)$ for every v ; and $L[v] = \infty$ exactly for the vertices not reachable from s , for which $\pi[v] = \text{null}$ and REPORTPATH correctly reports non-reachability;
- (ii) for every $v \neq s$ reachable from s , $\pi[v] \neq \text{null}$ and the recorded edge is tight: $L[v] = L[\pi[v]] + w(\pi[v], v)$;
- (iii) for every v reachable from s , the sequence $v, \pi[v], \pi[\pi[v]], \dots$ consists of distinct vertices, terminates at s after $k \leq n-1$ steps, and the reversed sequence is a shortest s - v path;
- (iv) REPORTPATH(v) runs in $O(k)$ time, where k is the number of edges of the reported path.

Proof. (i) The labels are computed exactly as in Bellman-Ford, whose correctness was established in Problem 2; a label is finite if and only if it is realised by a walk from s (Lemma 3.6), i.e. exactly for the reachable vertices, and $\pi[v]$ is assigned only together with a finite label. Note also that $L[s]$ and $\pi[s]$ are never modified: an update of $L[s]$ would require an edge (u, s) with $L[u] + w(u, s) < L[s] = 0$, and since $L[u]$ is the weight of an actual s - u walk, this would exhibit a closed walk through s of negative weight, hence a negative cycle.

(ii) Let $v \neq s$ be reachable. Then $L[v] < \infty$, so $L[v]$ was updated at least once and $\pi[v] \neq \text{null}$. Put $u = \pi[v]$. By Lemma 3.9, $L[v] \geq L[u] + w(u, v)$. By (i), $L[v] = \delta(s, v)$ and $L[u] = \delta(s, u)$, and the triangle inequality for shortest distances gives $\delta(s, v) \leq \delta(s, u) + w(u, v)$. The two inequalities are opposite, hence

$$L[v] = \delta(s, v) = \delta(s, u) + w(u, v) = L[\pi[v]] + w(\pi[v], v),$$

so the recorded edge is tight at termination. (Note that this conclusion is obtained from the inequality invariant together with the triangle inequality, not from any claim that the edge remained tight during the execution.)

(iii) By Lemma 3.10 the predecessor graph has no cycle, so the vertices $v, \pi[v], \pi[\pi[v]], \dots$ are pairwise distinct and the chain must stop; it stops only at a vertex with $\pi = \text{null}$, and by (ii) the only reachable vertex with $\pi = \text{null}$ is s . Distinctness bounds the number of steps by $n-1$. Writing the chain as $v = u_0, u_1, \dots, u_k = s$, each pair (u_{i+1}, u_i) is an edge of G , so the reversal $P : s = u_k, \dots, u_0 = v$ is a genuine directed path. Summing the tight equalities of (ii) along P telescopes to

$$w(P) = \sum_{i=0}^{k-1} w(u_{i+1}, u_i) = L[u_0] - L[u_k] = \delta(s, v) - 0 = \delta(s, v),$$

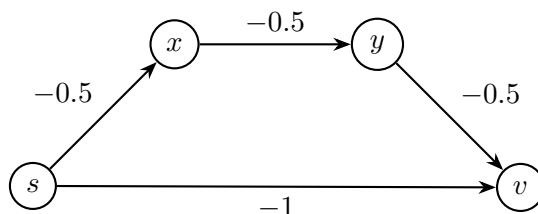
so P is a path of weight $\delta(s, v)$, i.e. a shortest s - v path.

(iv) REPORTPATH performs $O(1)$ work per pointer followed and k pointers are followed, so the time is $O(k)$, proportional to the number of edges of the reported shortest path, using the array π of size $O(n)$. $\square$

Problem 4: Adding a Constant to All Edge Weights Does Not Preserve Shortest Paths

Claim 3.12. *Let w be the most negative edge weight of G and let G' be obtained by adding $|w|$ to the weight of every edge. Then the shortest s - v paths of G' need not be shortest s - v paths of G . The student's claim is false.*

Counterexample. Let $V = \{s, x, y, v\}$ with the edges $s \rightarrow v$ of weight -1 , and $s \rightarrow x$, $x \rightarrow y$, $y \rightarrow v$, each of weight -0.5 .



The graph is acyclic, so it has no negative cycle. In G the direct path $s \rightarrow v$ has weight -1 , while the path $s \rightarrow x \rightarrow y \rightarrow v$ has weight $-0.5 - 0.5 - 0.5 = -1.5$. Since $-1.5 < -1$, the unique shortest s - v path in G is the three-edge path and $\delta(s, v) = -1.5$.

The most negative weight is -1 , so $|w| = 1$ is added to every edge. In G' the direct edge has weight $-1 + 1 = 0$ and each of the other three edges has weight $-0.5 + 1 = 0.5$. The direct path now has weight 0 and the three-edge path has weight 1.5 , so the unique shortest s - v path in G' is the direct edge. The shortest path has changed.

The reason is structural: if a path P has k edges and $C = |w|$ is added to every edge, then

$$w'(P) = w(P) + kC.$$

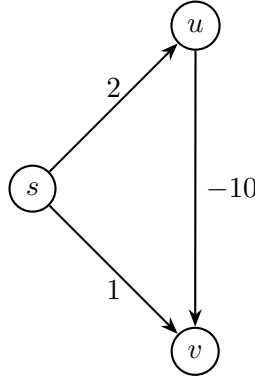
Paths with different numbers of edges are shifted by different amounts, so the transformation does not preserve the relative order of paths of unequal length, and hence cannot preserve shortest paths. (It does preserve them when all s - v paths under comparison have the same number of edges, which is not the case in general.) $\square$

Problem 5: True/False on Dijkstra and Bellman-Ford with Negative Weights

(a) FALSE. The claim is that for *every* directed graph, Dijkstra's algorithm fails as soon as *some* negative edge is present. A single graph in which a negative edge is present and Dijkstra is still correct refutes it.

Counterexample. Let $V = \{s, v, a, b\}$ with edges $s \rightarrow v$ of weight 5 and $a \rightarrow b$ of weight -100 , and no other edges. The negative edge is not reachable from s . Dijkstra from s finalises s with 0 , then v with 5 , and leaves a and b at ∞ . These are exactly the true distances $\delta(s, s) = 0$, $\delta(s, v) = 5$, $\delta(s, a) = \delta(s, b) = \infty$. So the presence of a negative edge does not by itself make Dijkstra fail; what matters is whether a negative edge can be used on a relevant path.

(b) **TRUE.** *Witness.* Let $V = \{s, u, v\}$ with $s \rightarrow v$ of weight 1, $s \rightarrow u$ of weight 2 and $u \rightarrow v$ of weight -10 .



The true distance is $\delta(s, v) = \min(1, 2 - 10) = -8$. Dijkstra initialises $\text{dist}[s] = 0$, extracts s and relaxes its edges, obtaining $\text{dist}[u] = 2$ and $\text{dist}[v] = 1$. The next extracted vertex is the one with the smallest tentative distance, namely v with value 1, and Dijkstra *finalises* it. Only afterwards is u extracted and the edge $u \rightarrow v$ relaxed, but v is already finalised, so the output keeps $\text{dist}[v] = 1 \neq -8$. Dijkstra therefore fails on this graph. (The failure is intrinsic to the algorithm's greedy invariant: it assumes no already-extracted vertex can be improved, which requires non-negative weights.)

(c) **TRUE.** We state the convention explicitly, because two versions of Bellman-Ford are in circulation.

- The *plain* algorithm performs $n - 1$ passes of relaxation and returns the resulting labels.
- The *augmented* algorithm performs one extra pass and reports the existence of a negative cycle if some edge can still be relaxed.

The statement asserts the existence of *one* graph with negative edges on which Bellman-Ford may fail to compute the distances, and this is true for the plain algorithm.

Example. Let $V = \{s, a, b\}$ with $s \rightarrow a$ of weight 1, $a \rightarrow b$ of weight 1 and $b \rightarrow a$ of weight -3 . The cycle $a \rightarrow b \rightarrow a$ has weight -2 and is reachable from s , so $\delta(s, a) = \delta(s, b) = -\infty$: there is no finite distance. The plain algorithm runs $n - 1 = 2$ passes and returns finite labels (which keep decreasing if more passes are performed), so its output is not the distance from s .

Remark. The augmented algorithm does not return a wrong number here: it detects the negative cycle and reports it. And if the graph has negative edges but no negative cycle, Bellman-Ford — plain or augmented — always computes the distances correctly, as proved in Problem 2. So the failure in (c) is caused by the negative *cycle*, not by negative edges as such.

Problem 6: Verifying a Candidate Distance Array

Task. $G = (V, E)$ has real edge weights and no negative cycle, and $s \in V$. Given an array A , verify whether $A[i] = \delta(s, i)$ for every $i \in V$, as efficiently as possible. Vertices not reachable from s have $\delta(s, i) = +\infty$; we allow the corresponding entries of A to carry the symbol $+\infty$, and we call an entry *finite* otherwise.

Algorithm 14: VerifyDistances(G, s, A)

```
1 if  $A[s] \neq 0$  then return "INCORRECT";
2 foreach edge  $(u, v) \in E$  with  $A[u]$  finite do
3   if  $A[v] = +\infty$  or  $A[v] > A[u] + w(u, v)$  then return "INCORRECT";
4  $T \leftarrow \{(u, v) \in E : A[u], A[v] \text{ finite and } A[v] = A[u] + w(u, v)\}$ ;
5 Run BFS (or DFS) from  $s$  in  $T$ ;
6 if some vertex  $v$  with  $A[v]$  finite is not reached then return "INCORRECT";
7 return "CORRECT";
```

Complexity. Each of the three checks is a single scan of the vertices and edges, and the search in T visits each vertex and edge at most once: $O(m + n)$ time. This is optimal up to constant factors, since any correct verifier must read essentially the whole input: if an edge (u, v) were never examined, its weight could be lowered enough to change $\delta(s, v)$ without the algorithm noticing, so its answer would be wrong on one of the two instances. Note also that this is strictly cheaper than recomputing the distances with Bellman-Ford, which costs $O(mn)$.

Theorem 3.13. *The algorithm outputs "CORRECT" if and only if $A[v] = \delta(s, v)$ for every $v \in V$.*

Proof. ($\Leftarrow$) Suppose $A = \delta(s, \cdot)$. Then $A[s] = 0$ since G has no negative cycle. For an edge (u, v) with $\delta(s, u)$ finite, v is reachable from s and the triangle inequality gives $\delta(s, v) \leq \delta(s, u) + w(u, v)$, so the second check passes. For every reachable $v \neq s$ there is a shortest s - v path, and all its edges are tight with respect to δ by optimal substructure, so this path lies inside T ; hence every vertex with a finite entry is reached from s in T and the third check passes.

($\Rightarrow$) Suppose all three checks pass. Let $F = \{v : A[v] \text{ finite}\}$; note $s \in F$ by the first check.

Every reachable vertex lies in F , and $A[v] \leq \delta(s, v)$ there. Let $P : s = x_0, \dots, x_k = v$ be any s - v path. The second check applied to (x_0, x_1) shows $x_1 \in F$ and $A[x_1] \leq A[x_0] + w(x_0, x_1)$; inductively every $x_i \in F$ and $A[x_i] \leq A[x_{i-1}] + w(x_{i-1}, x_i)$. Summing telescopically, $A[v] \leq A[s] + w(P) = w(P)$. Hence $v \in F$ and, minimising over all s - v paths, $A[v] \leq \delta(s, v)$.

Every vertex of F is reachable and $A[v] \geq \delta(s, v)$ there. By the third check there is a path $s = y_0, y_1, \dots, y_r = v$ all of whose edges lie in T . It is an actual path of G , so v is reachable, and summing the tight equalities telescopically gives $A[v] = A[s] + \sum_{i=1}^r w(y_{i-1}, y_i) = w(\text{that path}) \geq \delta(s, v)$.

Combining, F is exactly the set of vertices reachable from s , and $A[v] = \delta(s, v)$ for $v \in F$; for $v \notin F$ we have $A[v] = +\infty = \delta(s, v)$ because v is unreachable. Hence $A = \delta(s, \cdot)$. $\square$

Remark 3.14. The third check is exactly the additional condition identified in Problem 1: it is what rules out the pathological labelling of Proposition 3.2. In that instance the only tight edges are (a, b) and (b, a) , while (s, a) is not tight because $5 < 0 + 10$; hence a and b are not reachable from s inside T and the verifier correctly answers "INCORRECT". Note that the verifier needs no assumption about zero-weight cycles: it *tests* the grounding condition instead of assuming it.

Problem 7: Johnson's Algorithm for All-Pairs Shortest Paths

Setup. $G = (V, E)$ is strongly connected, with real edge weights and no negative cycle. We fill in the three steps sketched in the problem.

Step 1: a feasible potential. Pick an arbitrary vertex $s \in V$. Since G is strongly connected, every vertex is reachable from s , and since there is no negative cycle, Bellman-Ford computes $h(v) := \delta(s, v) \in \mathbb{R}$ for all v in $O(mn)$ time.

Step 2: reweighting. For every edge $(u, v) \in E$ define

$$w'(u, v) := w(u, v) + h(u) - h(v).$$

Lemma 3.15. $w'(u, v) \geq 0$ for every edge $(u, v) \in E$.

Proof. $h(v) = \delta(s, v) \leq \delta(s, u) + w(u, v) = h(u) + w(u, v)$ by the triangle inequality (a shortest s - u path followed by the edge (u, v) is an s - v walk, and with no negative cycle the distance is the minimum weight of a walk). Rearranging gives $w'(u, v) \geq 0$. $\square$

Lemma 3.16. For every $x, y \in V$ and every path P from x to y ,

$$w'(P) = w(P) + h(x) - h(y).$$

Consequently $\delta'(x, y) = \delta(x, y) + h(x) - h(y)$, and a path from x to y is w -shortest if and only if it is w' -shortest.

Proof. Let $P : x = z_0, z_1, \dots, z_k = y$. Then

$$w'(P) = \sum_{i=1}^k (w(z_{i-1}, z_i) + h(z_{i-1}) - h(z_i)) = w(P) + \sum_{i=1}^k (h(z_{i-1}) - h(z_i)) = w(P) + h(x) - h(y),$$

the second sum telescoping. The quantity $h(x) - h(y)$ depends only on the endpoints, not on P , so the w' -weights of all x - y paths are obtained from their w -weights by adding one and the same constant. Minimising therefore selects the same paths and $\delta'(x, y) = \delta(x, y) + h(x) - h(y)$. $\square$

Step 3: Dijkstra from every $v \in V \setminus \{s\}$. By Lemma 3.15 all w' -weights are non-negative, so Dijkstra's algorithm is applicable to (V, E, w') . The distances from s itself are already known from Step 1, so only the $n - 1$ sources in $V \setminus \{s\}$ need a Dijkstra run, exactly as the problem prescribes.

Step 4: recovering the original distances. For every pair (x, y) , Lemma 3.16 gives

$$\delta(x, y) = \delta'(x, y) - h(x) + h(y),$$

and a w' -shortest x - y path returned by Dijkstra is also a w -shortest x - y path.

Algorithm 15: JohnsonAPSP(G)

Input: Strongly connected $G = (V, E)$ with real weights w and no negative cycle.

Output: $\delta(x, y)$ for all $x, y \in V$.

- 1 Pick an arbitrary $s \in V$; run Bellman-Ford from s and set $h(v) \leftarrow \delta(s, v)$ for all v ;
// $O(mn)$
 - 2 **foreach** $(u, v) \in E$ **do** $w'(u, v) \leftarrow w(u, v) + h(u) - h(v)$;
 - 3 **foreach** $y \in V$ **do** $\delta(s, y) \leftarrow h(y)$;
 - 4 **foreach** $v \in V \setminus \{s\}$ **do**
 - 5 Run Dijkstra with a Fibonacci heap from v on (V, E, w') , obtaining $\delta'(v, \cdot)$;
// $O(m + n \log n)$
 - 6 **foreach** $y \in V$ **do** $\delta(v, y) \leftarrow \delta'(v, y) - h(v) + h(y)$;
 - 7 **return** all $\delta(x, y)$;
-

Complexity. Step 1 costs $O(mn)$; the reweighting costs $O(m)$; the $n - 1$ Dijkstra runs with a Fibonacci heap cost $O((n - 1)(m + n \log n)) = O(mn + n^2 \log n)$; the final conversion costs $O(n^2)$. The total is

$$O(mn) + O(mn + n^2 \log n) + O(n^2) = O(mn + n^2 \log n),$$

which is better than the $O(n^3)$ of Floyd-Warshall whenever $m = o(n^2 / \log n)$.

Theorem 3.17. *JohnsonAPSP computes $\delta(x, y)$ for all $x, y \in V$ in $O(mn + n^2 \log n)$ time.*

Proof. The potential h is correct by the correctness of Bellman-Ford (Problem 2), which applies because G has no negative cycle and, by strong connectivity, all vertices are reachable from s , so every $h(v)$ is a real number. Lemma 3.15 makes w' non-negative, so each Dijkstra run correctly computes $\delta'(v, \cdot)$. Lemma 3.16 converts these into the original distances, and the row $\delta(s, \cdot) = h(\cdot)$ is already available from Step 1. The running time was computed above. $\square$